

Alfido Tech – Customer Transactions & Behavior Analysis

✓ Objective

The objective of this analysis is to study customer transactions and behavior to identify customer segments, purchasing patterns, retention trends, and churn risks.

Key Areas of Analysis

- Data Cleaning
- Feature Engineering
- RFM Customer Segmentation
- Purchase Pattern Analysis
- Retention Analysis
- Churn Risk Analysis
- Actionable Business Recommendations

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

✓ 1. Load Dataset

```
df = pd.read_csv("ecommerce_customer_data_custom_ratios.csv")

df.head()
```

	Customer ID	Purchase Date	Product Category	Product Price	Quantity	Total Purchase Amount	Payment Method	Customer Age
0	46251	2020-09-08 09:38:32	Electronics	12	3	740	Credit Card	37
1	46251	2022-03-05 12:56:35	Home	468	4	2739	PayPal	37
2	46251	2022-05-23 18:18:01	Home	288	2	3196	PayPal	37
3	46251	2020-11-12 13:13:29	Clothing	196	1	3509	PayPal	37
4	13593	2020-11-27 17:55:11	Home	449	1	3452	Credit Card	49

```
df.shape
```

```
(250000, 13)
```

```
df.columns
```

```
Index(['Customer ID', 'Purchase Date', 'Product Category', 'Product Price',
      'Quantity', 'Total Purchase Amount', 'Payment Method', 'Customer Age',
      'Returns', 'Customer Name', 'Age', 'Gender', 'Churn'],
      dtype='object')
```

2. Data Cleaning

```
df.isnull().sum()
df.duplicated().sum()
df = df.drop_duplicates()
df["Purchase Date"] = pd.to_datetime(df["Purchase Date"])
df["Returns"] = df["Returns"].fillna(0)
df["YearMonth"] = df["Purchase Date"].dt.to_period("M").astype(str)
```

```
df["YearMonth"] = df["Purchase Date"].dt.to_period("M").astype(str)
```

3. Feature Engineering – RFM Analysis

```
snapshot_date = df["Purchase Date"].max() + pd.Timedelta(days=1)
```

```
rfm = df.groupby("Customer ID").agg(  
    Recency=("Purchase Date", lambda x: (snapshot_date - x.max()).days),  
    Frequency=("Purchase Date", "count"),  
    Monetary=("Total Purchase Amount", "sum")  
) .reset_index()
```

```
rfm.head()
```

	Customer ID	Recency	Frequency	Monetary	
0	1	58	1	3491	
1	2	299	3	7988	
2	3	89	8	22587	
3	4	127	4	8715	
4	5	171	8	12524	

Next steps:

[Generate code with rfm](#)[New interactive sheet](#)

✓ RFM Scoring

```
rfm["R_score"] = pd.qcut(  
    rfm["Recency"],  
    5,  
    labels=[5,4,3,2,1]  
) .astype(int)
```

```
rfm["F_score"] = pd.qcut(  
    rfm["Frequency"].rank(method="first"),  
    5,  
    labels=[1,2,3,4,5]  
) .astype(int)
```

```
rfm["M_score"] = pd.qcut(  
    rfm["Monetary"].rank(method="first"),  
    5,  
    labels=[1,2,3,4,5]  
) .astype(int)
```

```
rfm["RFM_Score"] = (  
    rfm["R_score"] +  
    rfm["F_score"] +  
    rfm["M_score"]  
)
```

✓ 4. Customer Segmentation

```
def segment_customer(row):  
  
    r = row["R_score"]  
    f = row["F_score"]  
    m = row["M_score"]  
  
    if r >= 4 and f >= 4 and m >= 4:  
        return "Champions"  
  
    elif r >= 4 and (f >= 3 or m >= 3):  
        return "Loyal / High Potential"  
  
    elif r >= 3 and f <= 3:  
        return "Recent / New"  
  
    elif r <= 2 and f >= 4:  
        return "At Risk - High Value"  
  
    elif r <= 2 and f <= 3:  
        return "Hibernating"  
  
    else:  
        return "Needs Attention"
```

```
rfm["Segment"] = rfm.apply(segment_customer, axis=1)
```

```
rfm["Segment"].value_counts()
```

	count
Segment	
Hibernating	14936
Recent / New	9556
Champions	8739
Loyal / High Potential	7147
At Risk – High Value	4890
Needs Attention	4405

dtype: int64

5. Customer Segment Profiling

```
segment_summary = rfm.groupby("Segment").agg(
    Customers=("Customer ID", "count"),
    Avg_Recency=("Recency", "mean"),
    Avg_Frequency=("Frequency", "mean"),
    Avg_Monetary=("Monetary", "mean")
).sort_values(
    "Customers",
    ascending=False
)
```

segment_summary

	Customers	Avg_Recency	Avg_Frequency	Avg_Monetary
Segment				
Hibernating	14936	533.069831	3.289167	8981.694831
Recent / New	9556	137.344182	3.476873	9100.766743
Champions	8739	61.306557	7.647557	21785.056299
Loyal / High Potential	7147	64.597314	5.015951	13040.007276
At Risk – High Value	4890	387.433129	6.846626	18566.397751
Needs Attention	4405	186.013167	7.148014	19491.245857

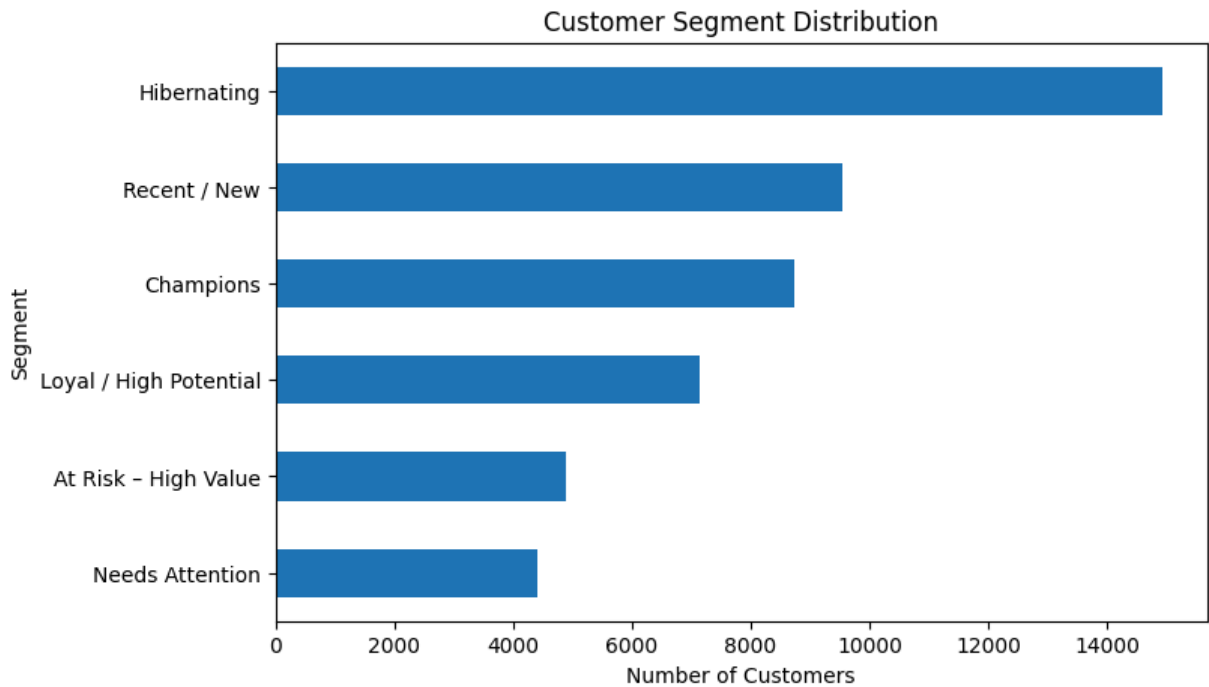
Next steps: [Generate code with segment_summary](#)

[New interactive sheet](#)

Customer Segment Distribution

```
segment_summary["Customers"].sort_values().plot(
    kind="barh",
    figsize=(8,5)
)

plt.title("Customer Segment Distribution")
plt.xlabel("Number of Customers")
plt.show()
```



6. Purchase Pattern Analysis

```
monthly = df.groupby("YearMonth").agg(
    Orders=("Customer ID", "size"),
    Customers=("Customer ID", "nunique"),
    Revenue=("Total Purchase Amount", "sum")
).reset_index()
```

```
plt.figure(figsize=(12,5))

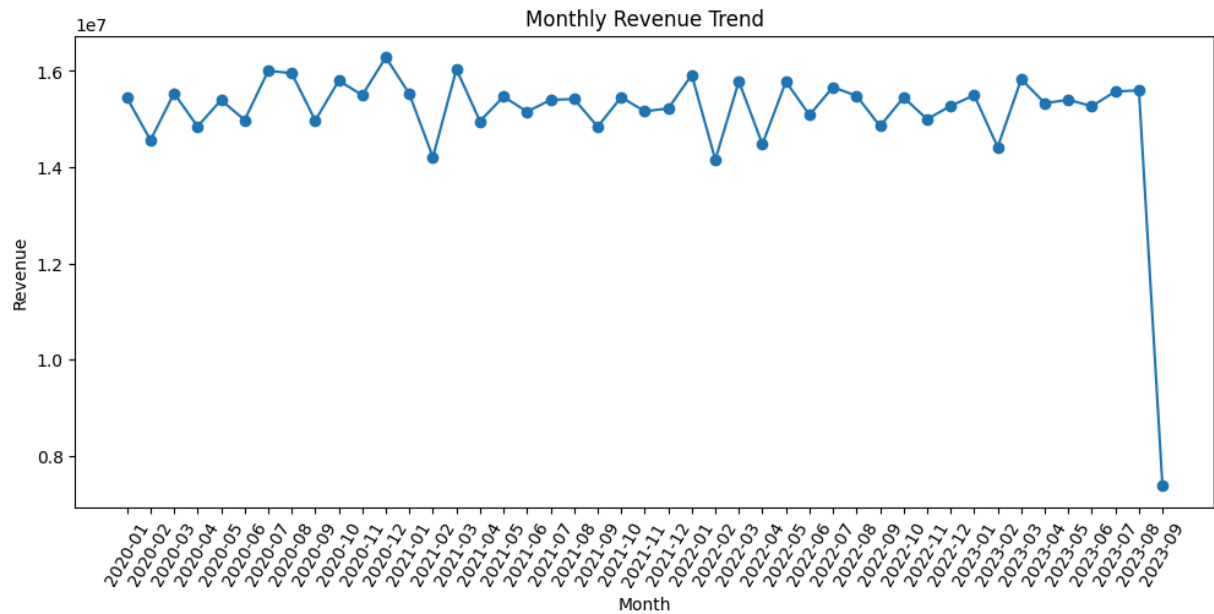
plt.plot(
    monthly["YearMonth"],
    monthly["Revenue"],
    marker="o"
```

```
)

plt.title("Monthly Revenue Trend")
plt.xlabel("Month")
plt.ylabel("Revenue")

plt.xticks(rotation=60)

plt.show()
```



```
category_summary = df.groupby(
    "Product Category"
).agg(
    Revenue=("Total Purchase Amount", "sum"),
    Orders=("Customer ID", "size"),
    Average_Order_Value=("Total Purchase Amount", "mean")
).sort_values(
    "Revenue",
    ascending=False
)

category_summary
```

	Revenue	Orders	Average_Order_Value	
Product Category				
Books	204939601	74912	2735.737946	
Clothing	204532405	75052	2725.209255	
Electronics	136599467	50185	2721.918243	
Home	135271210	49851	2713.510461	

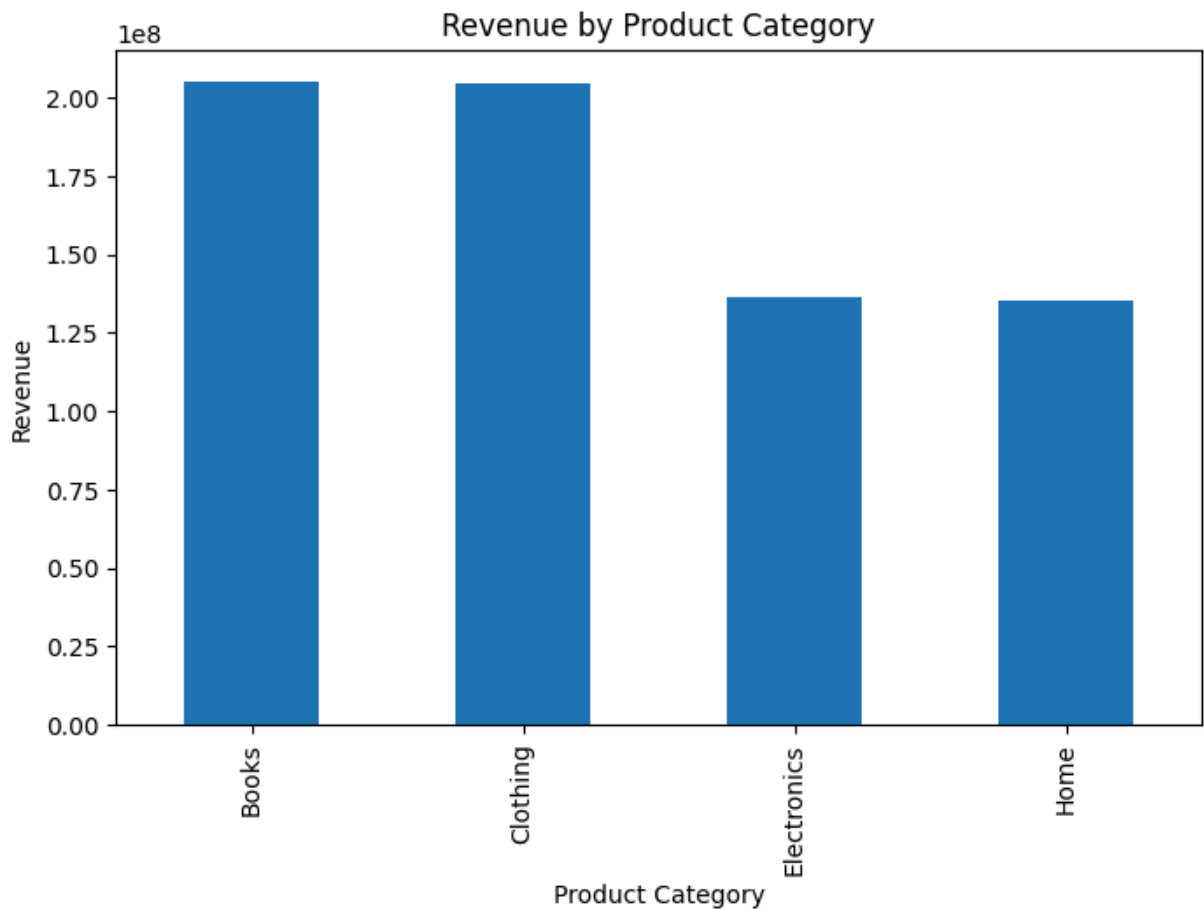
Next steps:

[Generate code with category_summary](#)[New interactive sheet](#)

```
category_summary["Revenue"].plot(
    kind="bar",
    figsize=(8,5)
)

plt.title("Revenue by Product Category")
plt.xlabel("Product Category")
plt.ylabel("Revenue")

plt.show()
```

7. Customer Retention Analysis

```
first_purchase = df.groupby(  
    "Customer ID"  
)[  
    "Purchase Date".min()  
]  
  
first_purchase.name = "FirstPurchase"
```

```
df_ret = df.merge(  
    first_purchase,  
    on="Customer ID"  
)
```

```
df_ret["IsRepeat"] = (  
    df_ret["Purchase Date"] >  
    df_ret["FirstPurchase"]  
)
```

```
retention = df_ret.groupby(
    "YearMonth"
).agg(
    Active_Customers=("Customer ID", "nunique"),
    Repeat_Customers=("IsRepeat", "sum")
).reset_index()
```

```
retention["Repeat_Customer_Rate"] = (
    retention["Repeat_Customers"] /
    retention["Active_Customers"]
)
```

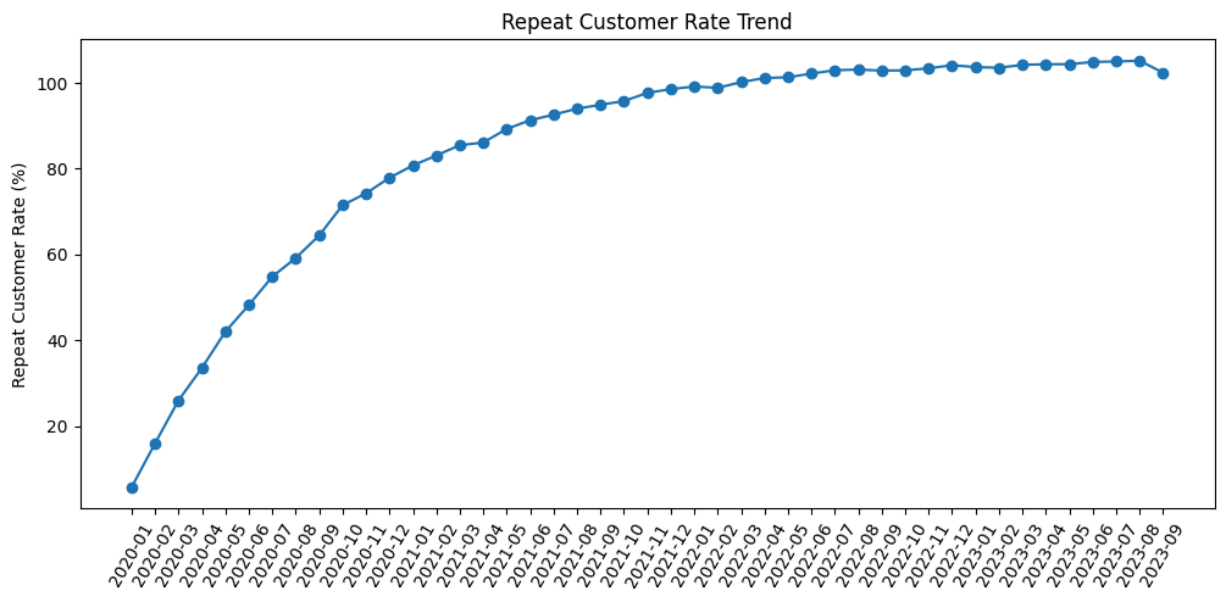
```
plt.figure(figsize=(12,5))

plt.plot(
    retention["YearMonth"],
    retention["Repeat_Customer_Rate"] * 100,
    marker="o"
)

plt.title("Repeat Customer Rate Trend")
plt.ylabel("Repeat Customer Rate (%)")

plt.xticks(rotation=60)

plt.show()
```



✓ 8. Churn Risk Analysis

```
overall_churn = df["Churn"].mean()

print(
    "Overall Churn Rate:",
    overall_churn * 100,
    "%"
)
```

Overall Churn Rate: 19.9496 %

```
segment_churn = rfm.merge(
    df[["Customer ID", "Churn"]].drop_duplicates("Customer ID"),
    on="Customer ID"
)
```

```
segment_churn.groupby("Segment")["Churn"].mean() * 100
```

	Churn
Segment	
At Risk – High Value	18.916155
Champions	19.659000
Hibernating	20.072309
Loyal / High Potential	20.120330
Needs Attention	20.839955
Recent / New	20.353704

dtype: float64

```
rfm["Churn_Risk"] = np.select(
    [
        rfm["Recency"] > 365,
        rfm["Recency"] > 180
    ],
    [
        "High",
        "Medium"
    ],
    default="Low"
)
```

```
rfm["Churn_Risk"].value_counts()
```

count	
Churn_Risk	
Low	24375
Medium	12724
High	12574

dtype: int64
Actionable Recommendations for Alfido Tech

✓ 1. Launch High-Value Win-Back Campaigns

Target the *At Risk – High Value* customers with personalized discounts, reminders, and product recommendations based on their previous purchases. This can help recover